

LABORATORIO DE PROGRAMACION

Interaccion con la API de Google Maps

desde una Cloud Function en Python

**LEE ESTE DOCUMENTO COMPLETO
ANTES DE ESCRIBIR CODIGO O ABRIR CUALQUIER CONSOLA.**

Objetivo. Invocar la API de Google Maps desde una Cloud Function escrita en Python y obtener datos geograficos de algun establecimiento. No se requiere interfaz grafica ni campos especificos: cualquier dato que la API devuelva es valido. Lo que importa es que la funcion desplegada en la nube haga la llamada y retorne la respuesta.

Flujo minimo esperado:

1. Llamas a tu Cloud Function via curl, Postman o Jupyter.
2. La funcion consulta la API de Google Maps.
3. La funcion retorna los datos geograficos como respuesta HTTP.

Modalidad. Trabajo en parejas. Ambos integrantes deben entender el codigo y la configuracion; cualquiera debe poder explicar el trabajo completo en la entrega.

Lenguaje. Todo el codigo debe estar en **Python**, tanto el prototipo en Jupyter como la Cloud Function final.

API Keys. Cada integrante del equipo debe generar su propia API Key en su propia cuenta de GCP. No se comparten keys entre companeros, ni siquiera con tu pareja.

Sobre esta guia. Los pasos de la pagina 4 son un punto de partida orientativo. No estan garantizados para funcionar sin adaptacion. Se espera que investigues, consultes la documentacion oficial referenciada y resuelvas los errores que aparezcan.

Advertencias

Tu API Key es una credencial privada

Una API Key autoriza cargos economicos en la cuenta que la genero. Tratala como una contraseña bancaria.

- **No la compartas con nadie**, incluyendo tu pareja. Cada integrante tiene su propia key en su propia cuenta.
- **No la escribas en el codigo**. Usa variables de entorno o Secret Manager.
- **No la subas a ningun repositorio**, publico ni privado.
- **No la envíes por ningun medio de comunicacion**: correo, WhatsApp, Slack, Teams, etc.
- Si sospechas que fue expuesta, **invalidala de inmediato** en *GCP Console > APIs & Services > Credentials*.

Riesgo de usar IA para resolver este laboratorio

Usar herramientas de IA (ChatGPT, Copilot, Claude, etc.) para resolver este laboratorio implica un riesgo real.

- Algunas herramientas pueden conectarse a servicios externos o ejecutar codigo. Si le das acceso a tu cuenta de GCP o pegas tu API Key en el chat, **la herramienta podria hacer llamadas a la nube en tu nombre sin que lo notes**, generando costos o comprometiendo tu cuenta.
- **Nunca pegues credenciales reales en una conversacion con una IA**. Los prompts pueden ser almacenados o procesados por terceros.
- Copiar una solucion sin entenderla va contra el objetivo del ejercicio y te dejara sin herramientas para el examen.

Cualquier cargo generado por una key mal configurada o expuesta es responsabilidad del alumno dueño de esa cuenta.

Recomendaciones

Mantener costo \$0.00

Google Maps Platform incluye \$200 USD de credito mensual gratuito, suficiente para cientos de llamadas de laboratorio. El riesgo no esta en el volumen sino en que la key quede expuesta. Estas medidas son suficientes para mantener el costo en cero:

1. **Restringe la key por IP** en *GCP Console > APIs & Services > Credentials*. Agrega solo la IP publica de tu maquina. Cualquier llamada desde otra IP sera rechazada aunque alguien tenga la key.
2. **Restringe la key a “Places API” unicamente.** Esto limita el dano si la key llega a filtrarse.
3. **Activa una alerta de presupuesto en \$0 USD** en *GCP > Billing > Budgets & alerts*. Recibiras un correo ante cualquier cargo antes de que escale. cloud.google.com/billing/docs/how-to/budgets

Flujo de trabajo sugerido

1. **Prototipa en Jupyter Notebook primero.** Llamadas directas desde tu maquina no generan costos de infraestructura. Valida que obtienes respuesta del API antes de desplegar cualquier cosa.
2. **Prueba la Cloud Function en el emulador local** antes del deploy. El emulador no consume cuota del API de Maps ni genera cargos.
3. **Despliega a la nube solo cuando el codigo ya funciona** en local.
4. **Haz llamadas de prueba con un conjunto pequeno de datos.** No necesitas cientos de llamadas para verificar que funciona.
5. **Lee los mensajes de error.** La consola de GCP y las respuestas del API son la fuente de verdad. Esa informacion te dira exactamente que hay que corregir.

Guia tecnica

Hoja de ruta sugerida — investiga lo que no entiendas

1. Configurar el proyecto en GCP

Crea un proyecto en GCP Console y habilita los servicios necesarios.

```
gcloud config set project TU_PROJECT_ID
gcloud services enable cloudfunctions.googleapis.com places-backend.
googleapis.com
```

developers.google.com/maps/get-started

2. Obtener y restringir la API Key

En *GCP Console > APIs & Services > Credentials > Create Credentials > API Key*.
Aplica restriccion de API (Places API) y restriccion de IP desde el inicio.

[Guia oficial de API Keys](#)

3. Prototipar en Jupyter Notebook

```
import requests, os

API_KEY = os.environ.get("MAPS_KEY") # exporta la variable antes de
    abrir Jupyter

resp = requests.get(
    "https://maps.googleapis.com/maps/api/place/findplacefromtext/json",
    params={
        "input": "Torre del Reformador Guatemala",
        "inputtype": "textquery",
        "fields": "place_id,name,geometry,formatted_address",
        "key": API_KEY,
    }
)
print(resp.json())
```

[Places API — Find Place from Text](#)

4. Manejar la API Key de forma segura

```
# Opcion A: variable de entorno al hacer deploy
gcloud functions deploy maps-query --set-env-vars MAPS_KEY=TU_KEY ...

# Opcion B: Secret Manager
echo -n "TU_KEY" | gcloud secrets create MAPS_KEY --data-file=-
```

[Variables de entorno en Cloud Functions](#)

5. Escribir la Cloud Function en Python

```
# main.py
import os, requests
import functions_framework

@functions_framework.http
def maps_query(request):
    place = request.args.get("place", "")
    if not place:
        return {"error": "parametro 'place' requerido"}, 400
    resp = requests.get(
        "https://maps.googleapis.com/maps/api/place/findplacefromtext/
        json",
        params={
            "input": place,
            "inputtype": "textquery",
            "fields": "place_id,name,geometry,formatted_address",
            "key": os.environ.get("MAPS_KEY"),
        }
    )
    return resp.json()
```

```
# requirements.txt
functions-framework==3.*
requests==2.*
```

[HTTP Functions en Python](#) · [Firebase Functions Gen~2](#)

6. Probar localmente y hacer deploy

```
# Prueba local (no consume cuota)
pip install functions-framework
functions-framework --target=maps_query --port=8080
curl "http://localhost:8080?place=Mercado+Central+Guatemala"

# Deploy
gcloud functions deploy maps-query \
    --gen2 --runtime=python312 --region=us-central1 \
    --source=. --entry-point=maps_query \
    --trigger-http --set-env-vars MAPS_KEY=TU_KEY
```

[Deploying Cloud Functions](#)

Entregable. Demostrar que al invocar la Cloud Function desplegada, esta retorna datos geograficos de la API de Google Maps. Los campos, el formato y el establecimiento quedan a criterio de la pareja.

Referencias adicionales

- [Google Maps Places API — Overview](#)
- [Google Maps — Precios y free tier](#)
- [GCP — Alertas de presupuesto](#)